Engineering at Anthropic Anthropic의 엔지니어링

Harness design for long-running application development

장기 실행 애플리케이션 개발을 위한 하네스 설계

Harness design is key to performance at the frontier of agentic coding. Here's how we pushed Claude further in frontend design and long-running autonomous software engineering.

하네스 설계는 에이전트 코딩의 최전선에서 성능의 핵심입니다. Claude가 프런트엔드 설계와 장기 실행 자율 소프트웨어 엔지니어링 분야에서 더욱 발전한 방법은 다음과 같습니다.

Published Mar 24, 2026

게시일: 2026년 3월 24일

Written by Prithvi Rajasekaran, a member of our Labs team.

Labs 팀의 일원인 *Prithvi Rajasekaran*이 작성했습니다.

Over the past several months I've been working on two interconnected problems: getting Claude to produce high-quality frontend designs, and getting it to build complete applications without human intervention. This work originated with earlier efforts on our frontend design skill and long-running coding agent harness, where my colleagues and I were able to improve Claude's performance well above baseline through prompt engineering and harness design—but both eventually hit ceilings.

지난 몇 달 동안 저는 Claude가 고품질 프런트엔드 디자인을 제작하도록 하는 것과 사람의 개입 없이 완전한 애플리케이션을 구축하도록 하는 두 가지 상호 연결된 문제를 해결하기 위해 노력해 왔습니다. 이 작업은 프런트엔드 설계 기술과 장기간 실행되는 코딩 에이전트 하네스에 대한 초기 노력에서 시작되었습니다. 동료와 저는 신속한 엔지니어링 및 하네스 설계를 통해 Claude의 성능을 기준보다 훨씬 더 향상시킬 수 있었지만 결국 둘 다 한계에 도달했습니다.

To break through, I sought out novel AI engineering approaches that held across two quite different domains, one defined by subjective taste, the other by verifiable correctness and usability. Taking inspiration from Generative Adversarial Networks (GANs), I designed a multi-agent structure with a **generator** and **evaluator** agent. Building an evaluator that graded outputs reliably—and with taste—meant first developing a set of criteria that could turn subjective judgments like "is this design good?" into concrete, gradable terms.

이를 돌파하기 위해 저는 주관적인 취향에 따라 정의되는 영역과 검증 가능한 정확성 및 유용성에 따라 정의되는 두 가지 매우 다른 영역을 포괄하는 새로운 AI 엔지니어링 접근 방식을 찾았습니다. GAN(Generative Adversarial Networks)에서 영감을 받아 생성자 에이전트와 평가자 에이전트가 있는 다중 에이전트 구조를 설계했습니다. 출력물을 안정적으로 평가하는 평가자를 구축한다는 것은 먼저 "이 디자인이 좋은가?"와 같은 주관적인 판단을 바꿀 수 있는 일련의 기준을 개발하는 것을 의미했습니다. 구체적이고 등급화 가능한 용어로 설명합니다.

I then applied these techniques to long-running autonomous coding, carrying over two lessons from our earlier harness work: decomposing the build into tractable chunks, and using structured artifacts to hand off context between sessions. The final result was a three-agent architecture—planner, generator, and evaluator—that produced rich full-stack applications over multi-hour autonomous coding sessions.

그런 다음 이러한 기술을 장기 실행 자율 코딩에 적용하여 이전 하네스 작업에서 두 가지 교훈을 얻었습니다. 즉, 빌드를 다루기 쉬운 덩어리로 분해하고 구조화된 아티팩트를 사용하여 세션 간에 컨텍스트를 전달했습니다. 최종 결과는 여러 시간의 자율 코딩 세션을 통해 풍부한 풀 스택 애플리케이션을 생성하는 3개의 에이전트 아키텍처(플래너, 생성기, 평가자)였습니다.

Why naive implementations fall short

순진한 구현이 부족한 이유

We've previously shown that harness design has a substantial impact on the effectiveness of long running agentic coding. In an earlier experiment, we used an initializer agent to decompose a product spec into a task list, and a coding agent that implemented the tasks one feature at a time before handing off artifacts to carry context across sessions. The broader developer community has converged on similar insights, with approaches like the "Ralph Wiggum" method using hooks or scripts to keep agents in continuous iteration cycles.

우리는 이전에 하네스 설계가 장기 실행 에이전트 코딩의 효율성에 상당한 영향을 미친다는 것을 보여주었습니다. 이전 실험에서 우리는 제품 사양을 작업 목록으로 분해하기 위해 초기화 에이전트를 사용했고, 세션 전반에 걸쳐 컨텍스트를 전달하기 위해 아티팩트를 전달하기 전에 한 번에 한 기능씩 작업을 구현하는 코딩 에이전트를 사용했습니다. 더 넓은 개발자 커뮤니티는 에이전트를 지속적인 반복 주기로 유지하기 위해 후크나 스크립트를 사용하는 "Ralph Wiggum" 방법과 같은 접근 방식을 통해 유사한 통찰력에 집중했습니다.

But some problems remained persistent. For more complex tasks, the agent still tends to go off the rails over time. While decomposing this issue, we observed two common failure modes with agents executing these sorts of tasks.

그러나 일부 문제는 여전히 지속되었습니다. 보다 복잡한 작업의 경우 에이전트는 시간이 지남에 따라 여전히 궤도를 벗어나는 경향이 있습니다. 이 문제를 분해하는 동안 우리는 이러한 종류의 작업을 실행하는 에이전트에서 두 가지 일반적인 실패 모드를 관찰했습니다.

First is that models tend to lose coherence on lengthy tasks as the context window fills (see our post on context engineering). Some models also exhibit "context anxiety," in which they begin wrapping up work prematurely as they approach what they believe is their context limit. Context resets—clearing the context window entirely and starting a fresh agent, combined with a structured handoff that carries the previous agent's state and the next steps—addresses both these issues.

첫 번째는 컨텍스트 창이 채워짐에 따라 모델이 긴 작업에서 일관성을 잃는 경향이 있다는 것입니다(컨텍스트 엔지니어링에 대한 게시물 참조). 일부 모델은 또한 자신의 상황 한계라고 믿는 수준에 접근하면서 작업을 조기에 마무리하기 시작하는 "상황 불안"을 나타냅니다. 컨텍스트 재설정(컨텍스트 창을 완전히 지우고 새로운 에이전트를 시작하는 것)은 이전 에이전트의 상태와 다음 단계를 전달하는 구조화된 핸드오프와 결합되어 이러한 문제를 모두 해결합니다.

This differs from compaction, where earlier parts of the conversation are summarized in place so the same agent can keep going on a shortened history. While compaction preserves continuity, it doesn't give the agent a clean slate, which means context anxiety can still persist. A reset provides a clean slate, at the cost of the handoff artifact having enough state for the next agent to pick up the work cleanly. In our earlier testing, we found Claude Sonnet 4.5 exhibited context anxiety strongly enough that compaction alone wasn't sufficient to enable strong long task performance, so context resets became essential to the harness design. This solves the core issue, but adds orchestration complexity, token overhead, and latency to each harness run.

이는 동일한 에이전트가 단축된 기록을 계속 진행할 수 있도록 대화의 이전 부분이 제자리에 요약되는 압축과 다릅니다. 압축은 연속성을 유지하지만 에이전트에 깨끗한 상태를 제공하지 않습니다. 이는 상황 불안이 여전히 지속될 수 있음을 의미합니다. 재설정은 다음 에이전트가 작업을 깔끔하게 선택할 수 있도록 충분한 상태를 갖는 핸드오프 아티팩트를 희생하여 깨끗한 슬레이트를 제공합니다. 이전 테스트에서 Claude Sonnet 4.5는 압축만으로는 강력한 장기 작업 성능을 구현하기에 충분하지 않을 정도로 강력한 컨텍스트 불안을 나타냈으므로 컨텍스트 재설정이 하네스 설계에 필수적이 되었습니다. 이는 핵심 문제를 해결하지만 각 하네스 실행에 오케스트레이션 복잡성, 토큰 오버헤드 및 대기 시간을 추가합니다.

A second issue, which we haven’t previously addressed, is self-evaluation. When asked to evaluate work they've produced, agents tend to respond by confidently praising the work—even when, to a human observer, the quality is obviously mediocre. This problem is particularly pronounced for subjective tasks like design, where there is no binary check equivalent to a verifiable software test. Whether a layout feels polished or generic is a judgment call, and agents reliably skew positive when grading their own work.

이전에 다루지 않은 두 번째 문제는 자체 평가입니다. 자신이 제작한 작업을 평가해 달라는 요청을 받으면 상담원은 자신 있게 작업을 칭찬하는 방식으로 응답하는 경향이 있습니다. 관찰자가 보기에 품질이 명백히 평범할 때에도 마찬가지입니다. 이 문제는 검증 가능한 소프트웨어 테스트와 동등한 바이너리 검사가 없는 설계와 같은 주관적인 작업에서 특히 두드러집니다. 레이아웃이 세련된지 일반적인지 여부는 판단에 달려 있으며 상담원은 자신의 작업을 채점할 때 확실히 긍정적인 편향을 보입니다.

However, even on tasks that do have verifiable outcomes, agents still sometimes exhibit poor judgment that impedes their performance while completing the task. Separating the agent doing the work from the agent judging it proves to be a strong lever to address this issue. The separation doesn't immediately eliminate that leniency on its own; the evaluator is still an LLM that is inclined to be generous towards LLM-generated outputs. But tuning a standalone evaluator to be skeptical turns out to be far more tractable than making a generator critical of its own work, and once that external feedback exists, the generator has something concrete to iterate against.

그러나 검증 가능한 결과가 있는 작업에서도 에이전트는 작업을 완료하는 동안 성능을 방해하는 잘못된 판단력을 보이는 경우가 있습니다. 작업을 수행하는 상담원과 이를 판단하는 상담원을 분리하면 이 문제를 해결하는 데 강력한 수단이 될 수 있습니다. 분리가 그 자체로 관대함을 즉시 제거하지는 않습니다. 평가자는 여전히 LLM에서 생성된 결과에 대해 관대한 경향이 있는 LLM입니다. 그러나 독립형 평가기를 회의적으로 조정하는 것은 생성기를 자체 작업에 비판적으로 만드는 것보다 훨씬 다루기 쉬운 것으로 밝혀졌으며 일단 외부 피드백이 존재하면 생성기는 반복할 구체적인 내용을 갖게 됩니다.

Frontend design: making subjective quality gradable

프론트엔드 디자인: 주관적인 품질 등급을 매길 수 있게 만들기

I started by experimenting on frontend design, where the self-evaluation issue was most visible. Absent any intervention, Claude normally gravitates toward safe, predictable layouts that are technically functional but visually unremarkable.

저는 자체 평가 문제가 가장 눈에 띄는 프런트엔드 디자인 실험부터 시작했습니다. 어떤 개입도 하지 않으면 Claude는 일반적으로 기술적으로 기능적이지만 시각적으로 눈에 띄지 않는 안전하고 예측 가능한 레이아웃을 선호합니다.

Two insights shaped the harness I built for frontend design. First, while aesthetics can’t be fully reduced to a score—and individual tastes will always vary—they can be improved with grading criteria that encode design principles and preferences. "Is this design beautiful?" is hard to answer consistently, but "does this follow our

principles for good design?" gives Claude something concrete to grade against. Second, by separating frontend generation from frontend grading, we can create a feedback loop that drives the generator toward stronger outputs.

두 가지 통찰력이 제가 프론트엔드 디자인을 위해 구축한 하네스를 형성했습니다. 첫째, 미학을 점수로 완전히 줄일 수는 없고 개인의 취향은 항상 다양하지만 디자인 원칙과 선호도를 반영하는 등급 기준을 통해 개선할 수 있습니다. “이 디자인이 예쁘나요?” 일관되게 대답하기는 어렵지만 "이것이 좋은 디자인에 대한 우리의 원칙을 따르는 것입니까?" Claude에게 채점할 구체적인 내용을 제공합니다. 둘째, 프런트엔드 생성과 프런트엔드 그레이딩을 분리함으로써 생성기를 더 강력한 출력으로 구동하는 피드백 루프를 생성할 수 있습니다.

With this in mind, I wrote four grading criteria that I gave to both the generator and evaluator agents in their prompts:

이를 염두에 두고 나는 생성자 에이전트와 평가자 에이전트 모두에게 프롬프트에 제공한 네 가지 채점 기준을 작성했습니다.

- **Design quality:** Does the design feel like a coherent whole rather than a collection of parts? Strong work here means the colors, typography, layout, imagery, and other details combine to create a distinct mood and identity.

  디자인 품질: 디자인이 부분의 집합이 아닌 일관된 전체처럼 느껴지나요? 여기서 강렬한 작업이란 색상, 타이포그래피, 레이아웃, 이미지 및 기타 세부 사항이 결합되어 독특한 분위기와 정체성을 만들어낸다는 것을 의미합니다.

- **Originality:** Is there evidence of custom decisions, or is this template layouts, library defaults, and AI-generated patterns? A human designer should recognize deliberate creative choices. Unmodified stock components—or telltale signs of AI generation like purple gradients over white cards—fail here.

  독창성: 맞춤형 결정의 증거가 있습니까? 아니면 이 템플릿 레이아웃, 라이브러리 기본값 및 AI 생성 패턴입니까? 인간 디자이너는 고의적이고 창의적인 선택을 인식해야 합니다. 수정되지 않은 기본 구성 요소 또는 흰색 카드 위의 보라색 그라데이션과 같은 AI 생성의 눈에 띄는 징후는 여기서 실패합니다.

- **Craft:** Technical execution: typography hierarchy, spacing consistency, color harmony, contrast ratios. This is a competence check rather than a creativity check. Most reasonable implementations do fine here by default; failing means broken fundamentals.

  제작: 기술적 실행: 타이포그래피 계층 구조, 간격 일관성, 색상 조화, 대비 비율. 창의성 점검이 아닌 역량 점검입니다. 가장 합리적인 구현은 기본적으로 여기에서 잘 수행됩니다. 실패는 기본이 무너졌다는 것을 의미합니다.

- **Functionality:** Usability independent of aesthetics. Can users understand what the interface does, find primary actions, and complete tasks without guessing?

  기능성: 미적 측면과 무관한 사용성. 사용자가 인터페이스의 기능을 이해하고, 기본 작업을 찾고, 추측하지 않고도 작업을 완료할 수 있습니까?

I emphasized design quality and originality over craft and functionality. Claude already scored well on craft and functionality by default, as the required technical competence tended to come naturally to the model. But on design and originality, Claude often produced outputs that were bland at best. The criteria explicitly penalized highly generic “AI slop” patterns, and by weighting design and originality more heavily it pushed the model toward more aesthetic risk-taking.

공예와 기능성보다 디자인 품질과 독창성을 강조했습니다. Claude는 필요한 기술 역량이 모델에 자연스럽게 적용되는 경향이 있었기 때문에 기본적으로 공예와 기능성에서 이미 좋은 점수를 받았습니다. 그러나 디자인과 독창성 측면에서 Claude는 종종 기껏해야 단조로운 결과물을 생산했습니다. 이 기준은 매우 일반적인 "AI 슬롭" 패턴에 명시적으로 불이익을 주었고, 디자인과 독창성에 더 큰 비중을 두어 모델이 더 미학적 위험을 감수하도록 밀어붙였습니다.

I calibrated the evaluator using few-shot examples with detailed score breakdowns. This ensured the evaluator’s judgment aligned with my preferences, and reduced score drift across iterations.

자세한 점수 분석이 포함된 몇 번의 예시를 사용하여 평가자를 보정했습니다. 이를 통해 평가자의 판단이 내 선호도와 일치하도록 보장하고 반복 전반에 걸쳐 점수 드리프트를 줄였습니다.

I built the loop on the Claude Agent SDK, which kept the orchestration straightforward. A generator agent first created an HTML/CSS/JS frontend based on a user prompt. I gave the evaluator the Playwright MCP, which let it interact with the live page directly before scoring each criterion and writing a detailed critique. In practice, the evaluator would navigate the page on its own, screenshotting and carefully studying the implementation before producing its assessment. That feedback flowed back to the generator as input for the next iteration. I ran 5 to 15 iterations per generation, with each iteration typically pushing the generator in a more distinctive direction as it responded to the evaluator's critique. Because the evaluator was actively navigating the page rather than scoring a static screenshot, each cycle took real wall-clock time. Full runs stretched up to four hours. I also instructed the generator to make a strategic decision after each evaluation: refine the current direction if scores were trending well, or pivot to an entirely different aesthetic if the approach wasn't working.

저는 Claude Agent SDK에 루프를 구축하여 오케스트레이션을 간단하게 유지했습니다. 생성기 에이전트는 먼저 사용자 프롬프트를 기반으로 HTML/CSS/JS 프런트엔드를 만들었습니다. 나는 평가자에게 Playwright MCP를 제공했는데, 이를 통해 각 기준을 채점하고 자세한 비평을 작성하기 전에 라이브 페이지와 직접 상호 작용할 수 있었습니다. 실제로 평가자는 평가를 작성하기 전에 자체적으로 페이지를 탐색하고 스크린샷을 찍고 구현을 주의 깊게 연구합니다. 해당 피드백은 다음 반복을 위한 입력으로 생성기로 다시 전달되었습니다. 나는 세대당 5~15번의 반복을 실행했으며, 각 반복은 일반적으로 평가자의 비평에 응답하면서 생성기를 보다 독특한 방향으로 밀어넣었습니다. 평가자는 정적 스크린샷에 점수를 매기기보다는 적극적으로 페이지를 탐색했기 때문에 각 주기에는 실제 벽시계 시간이 걸렸습니다. 전체 실행은 최대 4시간까지 연장됩니다. 또한 각 평가 후에 전략적 결정을 내리도록 생성자에게 지시했습니다. 즉, 점수 추세가 좋으면 현재 방향을 개선하고, 접근 방식이 작동하지 않으면 완전히 다른 미학으로 방향을 바꾸도록 지시했습니다.

Across runs, the evaluator's assessments improved over iterations before plateauing, with headroom still remaining. Some generations refined incrementally. Others took sharp aesthetic turns between iterations.

실행 전반에 걸쳐 평가자의 평가는 정체되기 전의 반복을 통해 개선되었으며 헤드룸은 여전히 남아 있습니다. 일부 세대는 점진적으로 개선되었습니다. 다른 사람들은 반복 사이에 날카로운 미적 변화를 겪었습니다.

The wording of the criteria steered the generator in ways I didn't fully anticipate. Including phrases like "the best designs are museum quality" pushed designs toward a particular visual convergence, suggesting that the prompting associated with the criteria directly shaped the character of the output.

기준의 표현은 내가 완전히 예상하지 못한 방식으로 발전기를 조종했습니다. "최고의 디자인은 박물관 품질이다"와 같은 문구를 포함하면 디자인이 특정 시각적 수렴 방향으로 밀렸으며, 이는 기준과 관련된 프롬프트가 출력물의 성격을 직접적으로 형성했음을 시사합니다.

While scores generally improved over iterations, the pattern was not always cleanly linear. Later implementations tended to be better as a whole, but I regularly saw cases where I preferred a middle iteration over the last one. Implementation complexity also tended to increase across rounds, with the generator reaching for more ambitious solutions in response to the evaluator’s feedback. Even on the first iteration, outputs were noticeably better than a baseline with no prompting at all, suggesting the criteria and associated language themselves steered the model away from generic defaults before any evaluator feedback led to further refinement.

점수는 일반적으로 반복을 통해 향상되었지만 패턴이 항상 완전히 선형인 것은 아닙니다. 이후 구현은 전체적으로 더 나은 경향이 있었지만 마지막 반복보다 중간 반복을 선호하는 경우를 정기적으로 보았습니다. 평가자의 피드백에 응답하여 생성자가 보다 야심찬 솔루션에 도달하면서 구현 복잡성도 라운드에 걸쳐 증가하는 경향이 있었습니다. 첫 번째 반복에서도 출력은 전혀 프롬프트가 표시되지 않은 기준선보다 눈에 띄게 좋았으며, 이는 평가자 피드백이 추가 개선으로 이어지기 전에 기준 및 관련 언어 자체가 모델을 일반적인 기본값에서 벗어나게 했음을 시사합니다.

In one notable example, I prompted the model to create a website for a Dutch art museum. By the ninth iteration, it had produced a clean, dark-themed landing page for a fictional museum. The page was visually polished but largely in line with my expectations. Then, on the tenth cycle, it scrapped the approach entirely and reimagined the site as a spatial experience: a 3D room with a checkered floor rendered in CSS perspective, artwork hung on the walls in free-form positions, and doorway-based navigation between gallery rooms instead of scroll or click. It was the kind of creative leap that I hadn't seen before from a single-pass generation.

한 가지 주목할만한 예에서는 모델에게 네덜란드 미술관용 웹사이트를 만들도록 요청했습니다. 9번째 반복에서는 가상 박물관을 위한 깨끗하고 어두운 테마의 랜딩 페이지를 제작했습니다. 페이지는 시각적으로 세련되었지만 대체로 내 기대와 일치했습니다. 그런 다음 10번째 주기에서는 접근 방식을 완전히 폐기하고 사이트를 공간적 경험으로 재구상했습니다. CSS 관점으로 렌더링된 체크무늬 바닥이 있는 3D 방, 자유로운 형식으로 벽에 걸린 예술 작품, 스크롤이나 클릭 대신 갤러리 방 사이의 출입구 기반 탐색이 가능했습니다. 이전 싱글 패스 세대에서는 볼 수 없었던 일종의 창의적인 도약이었습니다.

0:06 / 0:28

Scaling to full-stack coding

풀 스택 코딩으로 확장

With these findings in hand, I applied this GAN-inspired pattern to full-stack development. The generator-evaluator loop maps naturally onto the software development lifecycle, where code review and QA serve the same structural role as the design evaluator.

이러한 결과를 바탕으로 저는 GAN에서 영감을 받은 패턴을 풀 스택 개발에 적용했습니다. 생성기-평가자 루프는 코드 검토 및 QA가 설계 평가자와 동일한 구조적 역할을 하는 소프트웨어 개발 수명주기에 자연스럽게 매핑됩니다.

The architecture

아키텍처

In our earlier long-running harness, we had solved for coherent multi-session coding with an initializer agent, a coding agent that worked one feature at a time, and context resets between sessions. Context resets were a key unlock: the harness used Sonnet 4.5, which exhibited the “context anxiety” tendency mentioned earlier. Creating a harness that worked well across context resets was key to keeping the model on task. Opus 4.5 largely removed that behavior on its own, so I was able to drop context resets from this harness entirely. The agents were run as

one continuous session across the whole build, with the Claude Agent SDK's automatic compaction handling context growth along the way.

이전의 장기 실행 하네스에서는 초기화 에이전트, 한 번에 하나의 기능을 작동하는 코딩 에이전트 및 세션 간 컨텍스트 재설정을 사용하여 일관된 다중 세션 코딩을 해결했습니다. 컨텍스트 재설정은 중요한 잠금 해제였습니다. 하네스는 앞에서 언급한 "컨텍스트 불안" 경향을 나타내는 Sonnet 4.5를 사용했습니다. 컨텍스트 재설정 전반에 걸쳐 잘 작동하는 하네스를 만드는 것이 모델을 작업에 유지하는 데 핵심이었습니다. Opus 4.5는 자체적으로 이러한 동작을 대부분 제거했기 때문에 이 하네스에서 컨텍스트 재설정을 완전히 삭제할 수 있었습니다. 에이전트는 전체 빌드에서 하나의 연속 세션으로 실행되었으며, Claude Agent SDK의 자동 압축을 통해 컨텍스트 증가를 처리했습니다.

For this work I built on the foundation from the original harness with a three-agent system, with each agent addressing a specific gap I'd observed in prior runs. The system contained the following agent personas:

이 작업을 위해 저는 3개의 에이전트 시스템을 사용하여 원래 하네스의 기반을 구축했으며, 각 에이전트는 이전 실행에서 관찰한 특정 격차를 해결했습니다. 시스템에는 다음과 같은 상담원 페르소나가 포함되어 있습니다.

Planner: Our previous long-running harness required the user to provide a detailed spec upfront. I wanted to automate that step, so I created a planner agent that took a simple 1-4 sentence prompt and expanded it into a full product spec. I prompted it to be ambitious about scope and to stay focused on product context and high level technical design rather than detailed technical implementation. This emphasis was due to the concern that if the planner tried to specify granular technical details upfront and got something wrong, the errors in the spec would cascade into the downstream implementation. It seemed smarter to constrain the agents on the deliverables to be produced and let them figure out the path as they worked. I also asked the planner to find opportunities to weave AI features into the product specs. (See example in the Appendix at the bottom.)

플래너: 이전의 장기 실행 하네스에서는 사용자가 자세한 사양을 미리 제공해야 했습니다. 저는 그 단계를 자동화하고 싶었기 때문에 간단한 1~4개의 문장 프롬프트를 가져와 이를 전체 제품 사양으로 확장하는 플래너 에이전트를 만들었습니다. 나는 범위에 대한 야심찬 태도를 취하고 세부적인 기술 구현보다는 제품 컨텍스트와 높은 수준의 기술 설계에 계속 집중하도록 촉구했습니다. 이러한 강조는 기획자가 세부적인 기술 세부 사항을 미리 지정하려고 시도했지만 뭔가 잘못되었을 경우 사양의 오류가 다운스트림 구현에까지 영향을 미칠 수 있다는 우려 때문이었습니다. 상담원에게 산출물을 제한하고 작업하면서 경로를 파악하도록 하는 것이 더 현명해 보였습니다. 또한 기획자에게 AI 기능을 제품 사양에 접목할 수 있는 기회를 찾아달라고 요청했습니다. (하단 부록의 예시를 참고하세요.)

Generator: The one-feature-at-a-time approach from the earlier harness worked well for scope management. I applied a similar model here, instructing the generator to work in sprints, picking up one feature at a time from the spec. Each sprint implemented the app with a React, Vite, FastAPI, and SQLite (later PostgreSQL) stack, and the generator was instructed to self-evaluate its work at the end of each sprint before handing off to QA. It also had git for version control.

생성기: 이전 하네스의 한 번에 하나의 기능 접근 방식은 범위 관리에 적합했습니다. 여기에 비슷한 모델을 적용하여 생성기에 스프린트로 작동하도록 지시하고 사양에서 한 번에 하나의 기능을 선택했습니다. 각 스프린트는 React, Vite, FastAPI 및 SQLite(이후 PostgreSQL) 스택을 사용하여 앱을 구현했으며 생성기는 QA에 전달하기 전에 각 스프린트가 끝날 때 작업을 자체 평가하도록 지시 받았습니다. 버전 관리를 위한 git도 있었습니다.

Evaluator: Applications from earlier harnesses often looked impressive but still had real bugs when you actually tried to use them. To catch these, the evaluator used the Playwright MCP to click through the running application the way a user would, testing UI features, API endpoints, and database states. It then graded each sprint against both the bugs it had found and a set of criteria modeled on the frontend experiment, adapted here to cover product depth, functionality, visual design, and code quality. Each criterion had a hard threshold, and if any one fell below it, the sprint failed and the generator got detailed feedback on what went wrong.

Before each sprint, the generator and evaluator negotiated a sprint contract: agreeing on what "done" looked like for that chunk of work before any code was written. This existed because the product spec was intentionally high-level, and I wanted a step to bridge the gap between user stories and testable implementation. The generator proposed what it would build and how success would be verified, and the evaluator reviewed that proposal to make sure the generator was building the right thing. The two iterated until they agreed.

평가자: 이전 하네스의 응용 프로그램은 종종 인상적으로 보였지만 실제로 사용하려고 하면 여전히 실제 버그가 있었습니다. 이를 파악하기 위해 평가자는 Playwright MCP를 사용하여 사용자가 하는 방식으로 실행 중인 애플리케이션을 클릭하고 UI 기능, API 엔드포인트 및 데이터베이스 상태를 테스트했습니다. 그런 다음 발견한 버그와 제품 깊이, 기능, 시각적 디자인 및 코드 품질을 다루기 위해 여기에서 적용한 프런트엔드 실험에서 모델링된 일련의 기준에 대해 각 스프린트의 등급을 매겼습니다. 각 기준에는 엄격한 임계값이 있으며, 그 중 어느 하나라도 그 이하로 떨어지면 스프린트는 실패하고 생성기는 무엇이 잘못되었는지에 대한 자세한 피드백을 받습니다. 각 스프린트 전에 생성기와 평가자는 스프린트 계약을 협상했습니다. 코드가 작성되기 전에 해당 작업 덩어리에 대해 "완료"된 모습에 동의했습니다. 이는 제품 사양이 의도적으로 높은 수준이었기 때문에 발생했으며 사용자 스토리와 테스트 가능한 구현 간의 격차를 해소할 수 있는 단계를 원했습니다. 생성자는 무엇을 구축할지, 성공 여부를 어떻게 확인할 것인지를 제안했고, 평가자는 해당 제안을 검토하여 생성기가 올바른 것을 구축하고 있는지 확인했습니다. 두 사람은 동의할 때까지 반복했습니다.

Communication was handled via files: one agent would write a file, another agent would read it and respond either within that file or with a new file that the previous agent would read in turn. The generator then built against the agreed-upon contract before handing the work off to QA. This kept the work faithful to the spec without over-specifying implementation too early.

통신은 파일을 통해 처리되었습니다. 한 에이전트는 파일을 쓰고, 다른 에이전트는 파일을 읽고 해당 파일 내에서 응답하거나 이전 에이전트가 차례로 읽은 새 파일로 응답했습니다. 그런 다음 생성기는 작업을 QA에 넘기기 전에 합의된 계약에 따라 구축되었습니다. 이로 인해 너무 일찍 구현을 과도하게 지정하지 않고도 작업이 사양에 충실하게 유지되었습니다.

Running the harness

하네스 실행

For the first version of this harness, I used Claude Opus 4.5, running user prompts against both the full harness and a single-agent system for comparison. I used Opus 4.5 since this was our best coding model when I began these experiments.

이 하네스의 첫 번째 버전에서는 Claude Opus 4.5를 사용하여 비교를 위해 전체 하네스와 단일 에이전트 시스템 모두에 대해 사용자 프롬프트를 실행했습니다. 나는 이 실험을 시작했을 때 이것이 최고의 코딩 모델이었기 때문에 Opus 4.5를 사용했습니다.

I wrote the following prompt to generate a retro video game maker:

나는 복고풍 비디오 게임 제작자를 생성하기 위해 다음 프롬프트를 작성했습니다.

> *Create a 2D retro game maker with features including a level editor, sprite editor, entity behaviors, and a playable test mode.*

The table below shows the harness type, length it ran for, and the total cost.

아래 표에는 하네스 유형, 실행 길이 및 총 비용이 나와 있습니다.

Harness	Duration	Cost
Solo 독주	20 min 20분	$9 $9
Full harness 풀 하네스	6 hr 6시간	$200 $200

The harness was over 20x more expensive, but the difference in output quality was immediately apparent.

하네스는 가격이 20배 이상 높았지만 출력 품질의 차이는 즉각적으로 드러났습니다.

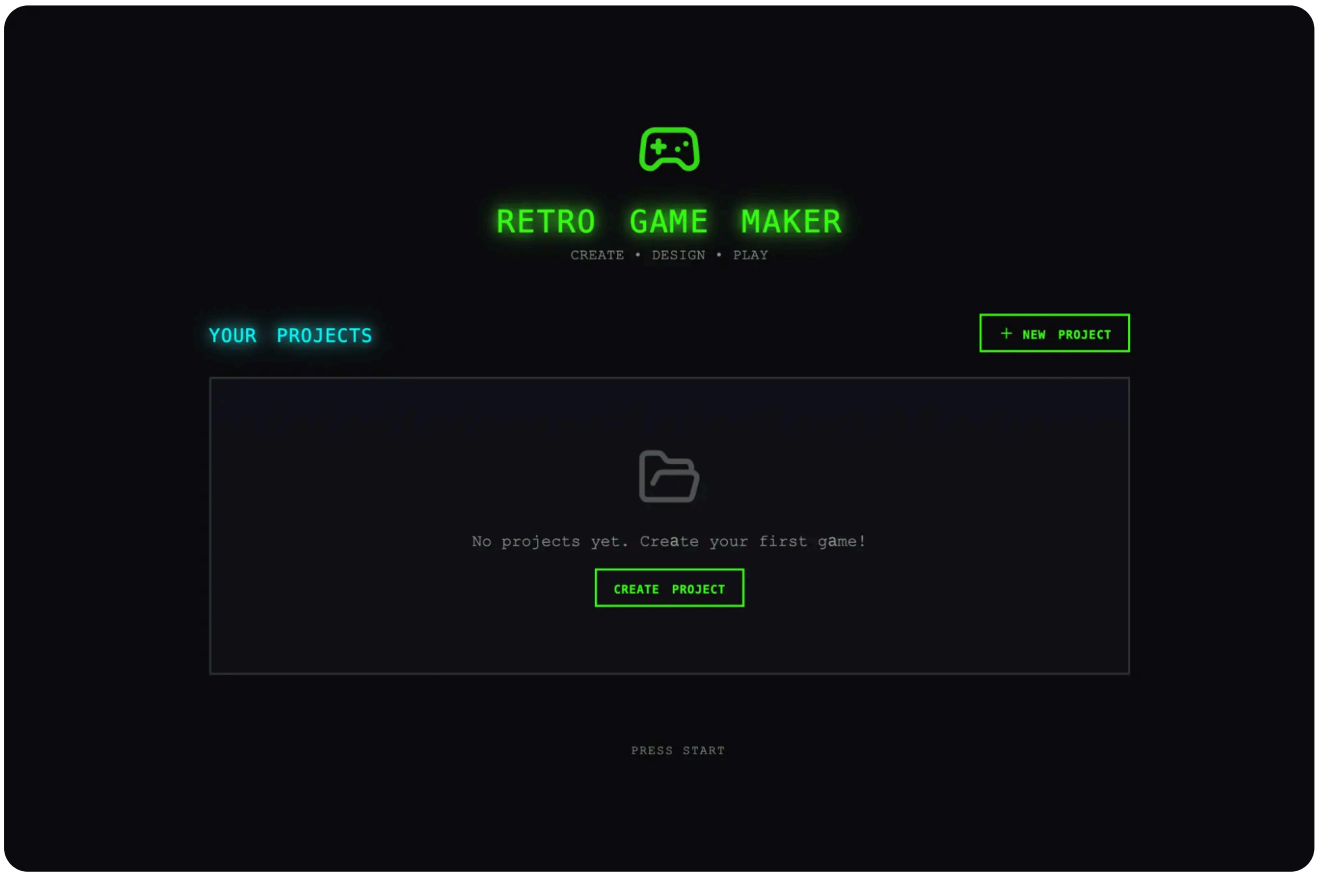
I was expecting an interface where I could construct a level and its component parts (sprites, entities, tile layout) then hit play to actually play the level. I started by opening the solo run's output, and the initial application seemed in line with those expectations.

저는 레벨과 해당 구성 요소(스프라이트, 엔터티, 타일 레이아웃)를 구성한 다음 재생 버튼을 눌러 실제로 레벨을 재생할 수 있는 인터페이스를 기대했습니다. 나는 솔로 런의 출력을 여는 것부터 시작했고 초기 애플리케이션은 그러한 기대와 일치하는 것처럼 보였습니다.

As I clicked through, however, issues started to emerge. The layout wasted space, with fixed-height panels leaving most of the viewport empty. The workflow was rigid. Trying to populate a level prompted me to create sprites and entities first, but nothing in the UI guided me toward that sequence. More to the point, the actual game was broken. My entities appeared on screen but nothing responded to input. Digging into the code revealed that the wiring between entity definitions and the game runtime was broken, with no surface indication of where.

그러나 클릭하면서 문제가 나타나기 시작했습니다. 레이아웃은 공간을 낭비했으며, 높이가 고정된 패널로 인해 대부분의 뷰포트가 비어 있었습니다. 작업 흐름은 엄격했습니다. 레벨을 채우려고 하면 먼저 스프라이트와 엔터티를 생성하라는 메시지가 표시되었지만 UI의 어떤 것도 해당 시퀀스로 안내하지 않았습니다. 더 중요한 것은 실제 게임이 망가졌다는 것입니다. 내 엔터티가 화면에 표시되었지만 입력에 응답하지 않았습니다. 코드를 자세히 살펴보면 엔터티 정의와 게임 런타임 사이의 연결이 손상되어 어디에 있는지 표면적으로 알 수 없다는 사실이 드러났습니다.

Opening screen 오프닝 화면 | Sprite editor 스프라이트 편집기 | Game play 게임 플레이



Initial screen when opening the app created by the solo harness.

솔로하네스에서 제작한 앱을 열었을 때의 초기화면입니다.

After evaluating the solo run, I turned my attention to the harness run. This run started from the same one-sentence prompt, but the planner step expanded that prompt into a 16-feature spec spread across ten sprints. It went well beyond what the solo run attempted. In addition to the core editors and play mode, the spec called for a sprite animation system, behavior templates, sound effects and music, an AI-assisted sprite generator and level designer, and game export with shareable links. I gave the planner access to our frontend design skill, which it read and used to create a visual design language for the app as part of the spec. For each sprint, the generator and evaluator negotiated a contract defining the specific implementation details for the sprint, and the testable behaviors that would be tested to verify completion.

솔로 런을 평가한 후 하네스 런에 관심을 돌렸습니다. 이 실행은 동일한 한 문장 프롬프트에서 시작되었지만 플래너 단계에서는 해당 프롬프트를 10개의 스프린트에 걸쳐 16개 기능 사양으로 확장했습니다. 그것은 솔로 런이 시도한 것보다 훨씬 뛰어났습니다. 핵심 편집기 및 플레이 모드 외에도 사양에는 스프라이트 애니메이션 시스템, 동작 템플릿, 음향 효과 및 음악, AI 지원 스프라이트 생성기 및 레벨 디자이너, 공유 가능한 링크가 있는 게임 내보내기가 필요했습니다. 나는 기획자에게 프런트엔드 디자인 기술에 대한 액세스 권한을 부여했고, 이를 읽고 사양의 일부로 앱의 시각적 디자인 언어를 만드는 데 사용했습니다. 각 스프린트에 대해 생성자와 평가자는 스프린트에 대한 구체적인 구현 세부 사항과 완료를 확인하기 위해 테스트할 테스트 가능한 동작을 정의하는 계약을 협상했습니다.

The app immediately showed more polish and smoothness than the solo run. The canvas used the full viewport, the panels were sized sensibly, and the interface had a consistent visual identity that tracked the design direction from the spec. Some of the clunkiness I'd seen in the solo run did remain—the workflow still didn't make it clear that you should build sprites and entities before trying to populate a level, and I had to figure that out by poking around. This read as a gap in the base model's product intuition rather than something the harness was designed to address, though it did suggest a place where targeted iteration inside the harness

could help to further improve output quality.

앱은 솔로 실행보다 즉시 더 세련되고 부드러움을 보여주었습니다. 캔버스는 전체 뷰포트를 사용했고 패널 크기는 합리적으로 조정되었으며 인터페이스는 사양의 디자인 방향을 추적하는 일관된 시각적 정체성을 가졌습니다. 단독 실행에서 보았던 투박함 중 일부는 그대로 남아 있습니다. 워크플로에서는 레벨을 채우기 전에 스프라이트와 엔터티를 빌드해야 한다는 점을 여전히 명확하게 밝히지 않았고, 저는 이 부분을 살펴보면서 알아내야 했습니다. 이는 하네스가 해결하도록 설계된 문제라기보다는 기본 모델의 제품 직관에 대한 격차로 해석되지만, 하네스 내부의 목표 반복이 출력 품질을 더욱 향상시키는 데 도움이 될 수 있는 위치를 제안했습니다.

Working through the editors, the new run's advantages over solo became more apparent. The sprite editor was richer and more fully featured, with cleaner tool palettes, a better color picker, and more usable zoom controls.

편집자를 통해 작업하면서 솔로에 비해 새로운 실행의 장점이 더욱 분명해졌습니다. 스프라이트 편집기는 더 깔끔한 도구 팔레트, 더 나은 색상 선택기, 더 유용한 확대/축소 컨트롤을 통해 더욱 풍부하고 모든 기능을 갖추고 있습니다.

Because I'd asked the planner to weave AI features into its specs, the app also came with a built-in Claude integration that let me generate different parts of the game through prompting. This significantly sped up the workflow.

기획자에게 AI 기능을 사양에 포함시켜 달라고 요청했기 때문에 앱에는 프롬프트를 통해 게임의 다양한 부분을 생성할 수 있는 Claude 통합 기능도 내장되어 있었습니다. 이로 인해 작업 흐름이 크게 빨라졌습니다.

Opening screen 오프닝 화면

Sprite editor 스프라이트 편집기

AI game design AI 게임 디자인

AI game design AI 게임 디자인

Game play 게임 플레이

Playing the game I generated

The biggest difference was in play mode. I was actually able to move my entity and play the game. The physics had some rough edges—my character jumped onto a platform but ended up overlapping with it, which felt intuitively wrong—but the core thing worked, which the solo run did not manage. After moving around a bit, I did hit some limitations with the AI's game level construction. There was a large wall that I wasn't able to jump past, so I was stuck. This suggested there were some common sense improvements and edge cases that the harness could handle to

further refine the app.

가장 큰 차이점은 플레이 모드에 있었습니다. 실제로 엔터티를 움직이고 게임을 할 수 있었습니다. 물리학에는 약간의 거친 부분이 있었습니다. 내 캐릭터가 플랫폼 위로 뛰어올랐지만 플랫폼과 겹쳐서 직관적으로 잘못된 느낌을 받았지만 핵심은 작동했지만 단독 실행에서는 관리되지 않았습니다. 약간 이동한 후 AI의 게임 레벨 구성에 몇 가지 제한 사항이 발생했습니다. 뛰어넘을 수 없는 큰 벽이 있어서 갇혀버렸어요. 이는 앱을 더욱 개선하기 위해 하네스가 처리할 수 있는 몇 가지 상식적인 개선 사항과 극단적인 경우가 있음을 시사했습니다.

Reading through the logs, it was clear that the evaluator kept the implementation in line with the spec. Each sprint, it walked through the sprint contract's test criteria and exercised the running application through Playwright, filing bugs against anything that diverged from expected behavior. The contracts were granular—Sprint 3 alone had 27 criteria covering the level editor—and the evaluator's findings were specific enough to act on without extra investigation. The table below shows several examples of issues our evaluator identified:

로그를 읽어보면 평가자가 사양에 맞게 구현을 유지했다는 것이 분명했습니다. 각 스프린트마다 스프린트 계약의 테스트 기준을 살펴보고 Playwright를 통해 실행 중인 애플리케이션을 실행하여 예상 동작에서 벗어나는 모든 항목에 대해 버그를 제출했습니다. 계약은 세부적이었습니다. Sprint 3에만 레벨 편집기를 포함하는 27가지 기준이 있었고 평가자의 결과는 추가 조사 없이 조치를 취할 수 있을 만큼 충분히 구체적이었습니다. 아래 표는 평가자가 식별한 문제의 몇 가지 예를 보여줍니다.

Contract criterion	Evaluator finding
Rectangle fill tool allows click-drag to fill a rectangular area with selected tile 직사각형 채우기 도구를 사용하면 클릭한 채 드래그하여 선택한 타일로 직사각형 영역을 채울 수 있습니다.	**FAIL** — Tool only places tiles at drag start/end points instead of filling the region. `fillRectangle` function exists but isn't triggered properly on mouseUp. FAIL — 도구는 영역을 채우는 대신 드래그 시작/끝 지점에만 타일을 배치합니다. `fillRectangle` 함수가 존재하지만 mouseUp에서 제대로 트리거되지 않습니다.
User can select and delete placed entity spawn points	

사용자는 배치된 개체 생성 지점을 선택하고 삭제할 수 있습니다.

FAIL — Delete key handler at `LevelEditor.tsx:892` requires both `selection` and `selectedEntityId` to be set, but clicking an entity only sets `selectedEntityId`. Condition should be `selection || (selectedEntityId && activeLayer === 'entity')`.

FAIL - `LevelEditor.tsx:892` 의 삭제 키 처리기에는 `selection` 과 `selectedEntityId` 모두 설정해야 하지만 엔터티를 클릭하면 `selectedEntityId` 만 설정됩니다. 조건은 `selection || (selectedEntityId && activeLayer === 'entity')`.

User can reorder animation frames via API

사용자는 API를 통해 애니메이션 프레임을 재정렬할 수 있습니다.

FAIL — `PUT /frames/reorder` route defined after `/{frame_id}` routes. FastAPI matches 'r`eorder`' as a frame_id integer and returns 422: "unable to parse string as an integer."

FAIL — `/{frame_id}` 경로 뒤에 정의된 `PUT /frames/reorder` 경로입니다. FastAPI는 'r`eorder`'를 프레임 ID 정수로 일치시키고 422를 반환합니다: "문자열을 정수로 구문 분석할 수 없습니다."

Getting the evaluator to perform at this level took work. Out of the box, Claude is a poor QA agent. In early runs, I watched it identify legitimate issues, then talk itself into deciding they weren't a big deal and approve the work anyway. It also tended to test superficially, rather than probing edge cases, so more subtle bugs often slipped through. The tuning loop was to read the evaluator's logs, find examples where its judgment diverged from mine, and update the QAs prompt to solve for those issues. It took several rounds of this development loop before the evaluator was grading in a way that I found reasonable. Even then, the harness output showed the limits of the model's QAing capabilities: small layout issues, interactions that felt unintuitive in places, and undiscovered bugs in more deeply nested features that the evaluator hadn't exercised thoroughly. There was clearly more verification headroom to capture with further tuning. But compared to the solo run, where the central feature of the application simply didn't work, the lift was obvious.

평가자가 이 수준에서 수행하도록 하는 데는 노력이 필요했습니다. 기본적으로 Claude는 품질이 좋지 않은 QA 에이전트입니다. 초기 실행에서 나는 그것이 합법적인 문제를 식별하는 것을 지켜본 다음 그것이 큰 문제가 아니라고 스스로 결정하고 어쨌든 작업을 승인했습니다. 또한 엣지 케이스를 조사하기보다는 피상적으로 테스트하는 경향이 있었기 때문에 더 미묘한 버그가 빠져나가는 경우가 많았습니다. 튜닝 루프는 평가자의 로그를 읽고, 평가자의 판단이 내 판단과 다른 예를 찾고, 이러한 문제를 해결하기 위해 QA 프롬프트를 업데이트하는 것이었습니다. 평가자가 내가 합리적이라고 생각하는 방식으로 등급을 매기려면 이 개발 루프를 여러 차례 반복해야 했습니다. 그럼에도 불구하고 하네스 출력은 모델의 QA 기능의 한계를 보여주었습니다. 작은 레이아웃 문제, 직관적이지 않은 상호 작용, 평가자가 철저하게 연습하지 않은 더 깊이 중첩된 기능에서 발견되지 않은 버그 등이 있었습니다. 추가 조정을 통해 포착할 수 있는 검증 여유가 분명히 더 많았습니다. 그러나 애플리케이션의 핵심 기능이 단순히 작동하지 않는 단독 실행과 비교하면 상승 효과는 분명했습니다.

Iterating on the harness

하네스에서 반복

The first set of harness results was encouraging, but it was also bulky, slow, and expensive. The logical next step was to find ways to simplify the harness without degrading its performance. This was partly common sense and partly a function of a more general principle: every component in a harness encodes an assumption about what the model can't do on its own, and those assumptions are worth stress testing, both because they may be incorrect, and because they can quickly go stale as models improve. Our blog post Building Effective Agents frames the underlying idea as "find the simplest solution possible, and only increase complexity when needed," and it's a pattern that shows up consistently for anyone maintaining an agent harness.

하네스 결과의 첫 번째 세트는 고무적이었지만 부피가 크고 느리며 비용이 많이 들었습니다. 논리적인 다음 단계는 성능을 저하시키지 않고 하네스를 단순화하는 방법을 찾는 것이었습니다. 이는 부분적으로는 상식이었으며 부분적으로는 보다 일반적인 원칙의 기능이었습니다. 하네스의 모든 구성 요소는 모델이 자체적으로 수행할 수 없는 작업에 대한 가정을 인코딩하며, 이러한 가정은 틀릴 수 있고 모델이 개선됨에 따라 빠르게 쓸모없어질 수 있기 때문에 스트레스 테스트할 가치가 있습니다. 우리 블로그 게시물 Building Effective Agents에서는 "가능한 가장 간단한 솔루션을 찾고 필요할 때만 복잡성을 증가시킨다"는 기본 아이디어를 제시하고 있으며 이는 에이전트 하네스를 유지 관리하는 모든 사람에게 일관되게 나타나는 패턴입니다.

In my first attempt to simplify, I cut the harness back radically and tried a few creative new ideas, but I wasn't able to replicate the performance of the original. It

also became difficult to tell which pieces of the harness design were actually load-bearing, and in what ways. Based on that experience, I moved to a more methodical approach, removing one component at a time and reviewing what impact it had on the final result.

단순화하려는 첫 번째 시도에서는 하네스를 근본적으로 줄이고 몇 가지 창의적인 새로운 아이디어를 시도했지만 원본의 성능을 재현할 수는 없었습니다. 또한 하네스 설계 중 어느 부분이 실제로 하중을 지탱하고 있는지, 어떤 방식으로 지탱하는지 파악하기가 어려워졌습니다. 그 경험을 바탕으로 저는 보다 체계적인 접근 방식으로 전환하여 한 번에 하나의 구성 요소를 제거하고 그것이 최종 결과에 어떤 영향을 미쳤는지 검토했습니다.

As I was going through these iteration cycles, we also released Opus 4.6, which provided further motivation to reduce harness complexity. There was good reason to expect 4.6 would need less scaffolding than 4.5 did. From our launch blog: "[Opus 4.6] plans more carefully, sustains agentic tasks for longer, can operate more reliably in larger codebases, and has better code review and debugging skills to catch its own mistakes." It also improved substantially on long-context retrieval. These were all capabilities the harness had been built to supplement.

이러한 반복 주기를 거치면서 우리는 하네스 복잡성을 줄이기 위한 추가 동기를 부여한 Opus 4.6도 출시했습니다. 4.6이 4.5보다 비계가 덜 필요할 것이라고 예상할 만한 충분한 이유가 있었습니다. 출시 블로그에서: "[Opus 4.6]은 더 신중하게 계획하고 에이전트 작업을 더 오랫동안 유지하며 더 큰 코드베이스에서 더 안정적으로 작동할 수 있으며 자체 실수를 포착할 수 있는 더 나은 코드 검토 및 디버깅 기술을 갖추고 있습니다." 또한 긴 컨텍스트 검색에서도 크게 개선되었습니다. 이는 하네스가 보완하기 위해 제작된 모든 기능이었습니다.

Removing the sprint construct

스프린트 구성 제거

I started by removing the sprint construct entirely. The sprint structure had helped to decompose work into chunks for the model to work coherently. Given the improvements in Opus 4.6, there was good reason to believe that the model could natively handle the job without this sort of decomposition.

나는 스프린트 구성을 완전히 제거하는 것부터 시작했습니다. 스프린트 구조는 모델이 일관되게 작동할 수 있도록 작업을 여러 덩어리로 분해하는 데 도움이 되었습니다. Opus 4.6의 개선 사항을 고려할 때 모델이 이러한 종류의 분해 없이 기본적으로 작업을 처리할 수 있다고 믿을 만한 충분한 이유가 있었습니다.

I kept both the planner and evaluator, as each continued to add obvious value. Without the planner, the generator under-scoped: given the raw prompt, it would

start building without first speccing its work, and end up creating a less feature-rich application than the planner did.

기획자와 평가자는 둘 다 계속해서 확실한 가치를 더해 주었기 때문에 그대로 두었습니다. 플래너가 없으면 생성기는 범위가 부족합니다. 원시 프롬프트가 주어지면 먼저 작업을 지정하지 않고 빌드를 시작하고 결국 플래너보다 기능이 덜 풍부한 애플리케이션을 생성하게 됩니다.

With the sprint construct removed, I moved the evaluator to a single pass at the end of the run rather than grading per sprint. Since the model was much more capable, it changed how load-bearing the evaluator was for certain runs, with its usefulness depending on where the task sat relative to what the model could do reliably on its own. On 4.5, that boundary was close: our builds were at the edge of what the generator could do well solo, and the evaluator caught meaningful issues across the build. On 4.6, the model's raw capability increased, so the boundary moved outward. Tasks that used to need the evaluator's check to be implemented coherently were now often within what the generator handled well on its own, and for tasks within that boundary, the evaluator became unnecessary overhead. But for the parts of the build that were still at the edge of the generator's capabilities, the evaluator continued to give real lift.

스프린트 구성을 제거한 후 평가자를 스프린트별로 채점하는 대신 실행이 끝날 때 단일 패스로 이동했습니다. 모델의 능력이 훨씬 더 뛰어나기 때문에 특정 실행에 대해 평가자가 부하를 견디는 방식이 변경되었으며 모델이 자체적으로 안정적으로 수행할 수 있는 작업과 관련하여 작업이 어디에 있는지에 따라 유용성이 달라졌습니다. 4.5에서는 그 경계가 가까웠습니다. 우리 빌드는 생성기가 단독으로 잘 수행할 수 있는 작업의 가장자리에 있었고 평가자는 빌드 전반에 걸쳐 의미 있는 문제를 포착했습니다. 4.6에서는 모델의 원시 성능이 증가하여 경계가 바깥쪽으로 이동했습니다. 일관되게 구현되기 위해 평가자의 확인이 필요했던 작업은 이제 생성기가 자체적으로 잘 처리하는 범위 내에 있는 경우가 많았으며 해당 경계 내의 작업에 대해 평가자는 불필요한 오버헤드가 되었습니다. 그러나 아직 발전기 성능의 가장자리에 있는 빌드 부분에 대해서는 평가자가 계속해서 실질적인 향상을 주었습니다.

The practical implication is that the evaluator is not a fixed yes-or-no decision. It is worth the cost when the task sits beyond what the current model does reliably solo.

실질적인 의미는 평가자가 고정된 예 또는 아니오 결정이 아니라는 것입니다. 현재 모델이 안정적으로 단독으로 수행하는 작업을 넘어서는 작업이 수행되면 비용을 지불할 가치가 있습니다.

Alongside the structural simplification, I also added prompting to improve how the harness built AI features into each app, specifically getting the generator to build a

proper agent that could drive the app's own functionality through tools. That took real iteration, since the relevant knowledge is recent enough that Claude's training data covers it thinly. But with enough tuning, the generator was building agents correctly.

구조적 단순화와 함께 하네스가 각 앱에 AI 기능을 구축하는 방식을 개선하라는 메시지도 추가했습니다. 특히 생성기가 도구를 통해 앱 자체 기능을 구동할 수 있는 적절한 에이전트를 구축하도록 했습니다. 관련 지식이 Claude의 훈련 데이터에서 다루는 내용이 얄팍할 정도로 최신 지식이었기 때문에 실제 반복 작업이 필요했습니다. 그러나 충분한 튜닝을 통해 생성기는 에이전트를 올바르게 구축했습니다.

Results from the updated harness

업데이트된 하네스의 결과

To put the updated harness to the test, I used the following prompt to generate a Digital Audio Workstation (DAW), a music production program for composing, recording, and mixing songs:

업데이트된 하네스를 테스트하기 위해 다음 프롬프트를 사용하여 노래 작곡, 녹음 및 믹싱을 위한 음악 제작 프로그램인 DAW(Digital Audio Workstation)를 생성했습니다.

> *Build a fully featured DAW in the browser using the Web Audio API.*

The run was still lengthy and expensive, at about 4 hours and $124 in token costs.

실행은 약 4시간과 토큰 비용 $124로 여전히 길고 비용이 많이 들었습니다.

Most of the time went to the builder, which ran coherently for over two hours without the sprint decomposition that Opus 4.5 had needed.

대부분의 시간은 Opus 4.5에 필요했던 스프린트 분해 없이 2시간 이상 일관되게 실행되는 빌더에 갔습니다.

Agent & Phase 에이전트 및 단계	Duration 지속	Cost 비용
Planner 입안자		

4.7 min 4.7분
$0.46 $0.46

Build (Round 1) 빌드(1라운드)
2 hr 7 min
2시간 7분
$71.08 $71.08

QA (Round 1) 품질보증(1라운드)
8.8 min 8.8분
$3.24 $3.24

Build (Round 2) 빌드(2라운드)
1 hr 2 min
1시간 2분
$36.89 $36.89

QA (Round 2) 품질보증(2라운드)
6.8 min 6.8분
$3.09 $3.09

Build (Round 3) 빌드(3라운드)
10.9 min 10.9분
$5.88 $5.88

QA (Round 3) 품질보증(3차)
9.6 min 9.6분
$4.06 $4.06

Total V2 Harness 총 V2 하네스
3 hr 50 min
3시간 50분
$124.70 $124.70

As with the previous harness, the planner expanded the one-line prompt into a full spec. From the logs, I could see the generator model did a good job planning the app and the agent design, wiring the agent up, and testing it before handing off to QA.

이전 하네스와 마찬가지로 플래너는 한 줄 프롬프트를 전체 사양으로 확장했습니다. 로그에서 생성기 모델이 앱과 에이전트 설계를 계획하고, 에이전트를 연결하고, QA에 전달하기 전에 테스트하는 일을 잘 수행했다는 것을 알 수 있었습니다.

That being said, the QA agent still caught real gaps. In its first-round feedback, it noted:

그럼에도 불구하고 QA 에이전트는 여전히 실제 격차를 포착했습니다. 1차 피드백에서 다음과 같이 언급했습니다.

> This is a strong app with excellent design fidelity, solid AI agent, and good backend. The main failure point is Feature Completeness — while the app looks impressive and the AI integration works well, several core DAW features are display-only without interactive depth: clips can't be dragged/moved on the timeline, there are no instrument UI panels (synth knobs, drum pads), and no visual effect editors (EQ curves, compressor meters). These aren't edge cases — they're the core interactions that make a DAW usable, and the spec explicitly calls for them.

In its second round feedback, it again caught several functionality gaps:

두 번째 피드백에서는 몇 가지 기능적 격차를 다시 발견했습니다.

> Remaining gaps:
> - Audio recording is still stub-only (button toggles but no mic capture)
> - Clip resize by edge drag and clip split not implemented
> - Effect visualizations are numeric sliders, not graphical (no EQ curve)

The generator was still liable to miss details or stub features when left to its own devices, and the QA still added value in catching those last mile issues for the generator to fix.

생성기는 자체 장치에 남겨두면 세부 사항이나 스텁 기능을 놓칠 가능성이 여전히 높았으며, QA는 생성기가 해결해야 할 라스트 마일 문제를 포착하는 데 여전히 가치를 더했습니다.

Based on the prompt, I was expecting a program where I could create melodies, harmonies, and drum patterns, arrange them into a song, and get help from an

integrated agent along the way. The video below shows the result.

프롬프트를 바탕으로 멜로디, 하모니, 드럼 패턴을 만들어서 노래로 편곡하고, 그 과정에서 통합 에이전트의 도움을 받을 수 있는 프로그램을 기대하고 있었습니다. 아래 영상은 그 결과를 보여줍니다.

0:30 / 0:30

The app is far from a professional music production program, and the agent's song composition skills could clearly use a lot of work. Additionally, Claude can't actually hear, which made the QA feedback loop less effective with respect to musical taste.

이 앱은 전문적인 음악 제작 프로그램과는 거리가 멀고, 에이전트의 작곡 능력은 분명히 많은 작업을 할 수 있을 것입니다. 게다가 Claude는 실제로 들을 수 없기 때문에 QA 피드백 루프가 음악적 취향에 비해 덜 효과적이었습니다.

But the final app had all the core pieces of a functional music production program: a working arrangement view, mixer, and transport running in the browser. Beyond that, I was able to put together a short song snippet entirely through prompting: the agent set the tempo and key, laid down a melody, built a drum track, adjusted mixer levels, and added reverb. The core primitives for song composition were

present, and the agent could drive them autonomously, using tools to create a simple production from end to end. You might say it's not pitch-perfect yet—but it's getting there.

그러나 최종 앱에는 기능적인 음악 제작 프로그램의 핵심 부분인 작업 편곡 보기, 믹서 및 브라우저에서 실행되는 전송 기능이 모두 포함되어 있었습니다. 그 외에도 에이전트가 템포와 키를 설정하고, 멜로디를 배치하고, 드럼 트랙을 만들고, 믹서 레벨을 조정하고, 리버브를 추가하는 등 프롬프트를 통해 짧은 노래 스니펫을 구성할 수 있었습니다. 노래 작곡을 위한 핵심 기본 요소가 존재했고, 에이전트는 도구를 사용하여 처음부터 끝까지 간단한 프로덕션을 생성하는 방식으로 이를 자율적으로 구동할 수 있었습니다. 아직은 완벽하지 않다고 말할 수도 있지만, 점점 그 단계에 도달하고 있습니다.

What comes next

다음에 무엇이 올 것인가?

As models continue to improve, we can roughly expect them to be capable of working for longer, and on more complex tasks. In some cases, that will mean the scaffold surrounding the model matters less over time, and developers can wait for the next model and see certain problems solve themselves. On the other hand, the better the models get, the more space there is to develop harnesses that can achieve complex tasks beyond what the model can do at baseline.

모델이 계속해서 개선됨에 따라 우리는 모델이 더 오래, 더 복잡한 작업을 수행할 수 있을 것이라고 대략적으로 기대할 수 있습니다. 어떤 경우에는 모델을 둘러싼 비계가 시간이 지남에 따라 덜 중요해지고 개발자는 다음 모델을 기다리면서 특정 문제가 스스로 해결되는 것을 볼 수 있습니다. 반면, 모델이 향상될수록 모델이 기본적으로 수행할 수 있는 것 이상으로 복잡한 작업을 수행할 수 있는 하네스를 개발할 수 있는 공간이 더 많아집니다.

With this in mind, there are a few lessons from this work worth carrying forward. It is always good practice to experiment with the model you're building against, read its traces on realistic problems, and tune its performance to achieve your desired outcomes. When working on more complex tasks, there is sometimes headroom from decomposing the task and applying specialized agents to each aspect of the problem. And when a new model lands, it is generally good practice to re-examine a harness, stripping away pieces that are no longer load-bearing to performance and adding new pieces to achieve greater capability that may not have been possible before.

이를 염두에 두고 이 작업에는 계속해서 추진할 가치가 있는 몇 가지 교훈이 있습니다. 구축 중인 모델을 실험하고, 현실적인 문제에 대한 흔적을 읽고, 성능을 조정하여 원하는 결과를 얻는 것은 항상 좋은 습관입니다. 더 복잡한 작업을 수행할 때 작업을 분해하고 문제의 각 측면에 전문 에이전트를 적용하는 데 여유 공간이 있는 경우가 있습니다. 그리고 새 모델이 출시되면 하네스를 다시 검사하여 더 이상 성능에 지탱되지 않는 부분을 제거하고 새 부분을 추가하여 이전에는 불가능했던 더 큰 기능을 달성하는 것이 일반적으로 좋은 습관입니다.

From this work, my conviction is that the space of interesting harness combinations doesn't shrink as models improve. Instead, it moves, and the interesting work for AI engineers is to keep finding the next novel combination.

이 작업을 통해 모델이 개선되더라도 흥미로운 하네스 조합의 공간이 줄어들지 않는다는 확신이 들었습니다. 대신 움직이며 AI 엔지니어의 흥미로운 작업은 계속해서 다음의 새로운 조합을 찾는 것입니다.

Acknowledgements

감사의 말

Special thanks to Mike Krieger, Michael Agaby, Justin Young, Jeremy Hadfield, David Hershey, Julius Tarng, Xiaoyi Zhang, Barry Zhang, Orowa Sidker, Michael Tingley, Ibrahim Madha, Martina Long, and Canyon Robbins for their contributions to this work.

이 작업에 기여한 Mike Krieger, Michael Agaby, Justin Young, Jeremy Hadfield, David Hershey, Julius Tarng, Xiaoyi Zhang, Barry Zhang, Orowa Sidker, Michael Tingley, Ibrahim Madha, Martina Long 및 Canyon Robbins에게 특별히 감사드립니다.

Thanks also to Jake Eaton, Alyssa Leonard, and Stef Sequeira for their help shaping the post.

게시물 작성에 도움을 주신 Jake Eaton, Alyssa Leonard 및 Stef Sequeira에게도 감사드립니다.

Appendix

Appendix

Example plan generated by planner agent.

플래너 에이전트가 생성한 계획 예시입니다.

```
RetroForge - 2D Retro Game Maker

Overview
RetroForge is a web-based creative studio for designing and
building 2D retro-style video games. It combines the nostalgic
charm of classic 8-bit and 16-bit game aesthetics with modern,
intuitive editing tools—enabling anyone from hobbyist creators to
indie developers to bring their game ideas to life without writing
traditional code.

The platform provides four integrated creative modules: a tile-
based Level Editor for designing game worlds, a pixel-art Sprite
Editor for crafting visual assets, a visual Entity Behavior system
for defining game logic, and an instant Playable Test Mode for
```

Copy Expand

Get the developer newsletter

개발자 뉴스레터 받기

Product updates, how-tos, community spotlights, and more. Delivered monthly to your inbox.

제품 업데이트, 방법, 커뮤니티 스포트라이트 등을 확인하세요. 매월 받은 편지함으로 배달됩니다.

Enter your email

Please provide your email address if you'd like to receive our monthly developer newsletter. You can unsubscribe at any time.

Customer support

Education

Financial services

Government

Healthcare

Life sciences

Nonprofits

Security

Claude Platform

Overview

Developer docs

Pricing

Marketplace

Regional compliance

Amazon Bedrock

Google Cloud's Vertex AI

Microsoft Foundry

Console login

Resources

Blog

Claude partner network

Community

Connectors

Courses

Customer stories

Engineering at Anthropic

Events

Inside Claude Code

Inside Claude Cowork

Plugins

Powered by Claude

Service partners

Startups program

Tutorials

Use cases

Help and security

Availability

Status

Support center

Company

Anthropic

Careers

Economic Futures

Research

News

Claude's Constitution

Responsible Scaling Policy

Security and compliance

Transparency

Terms and policies

Privacy choices

Privacy policy

Consumer health data privacy policy

Responsible disclosure policy

Terms of service: Commercial

Terms of service: Consumer

Usage policy

© 2026 Anthropic PBC